Epistemic Security via Indeterminate Emergence

Theory and Application of Absence-Preserving Action Systems

Adam Bishop
XOps360 LLC
ORCID: 0009-0000-4569-3726

March 2026

Preprint — submitted to IACR Cryptology ePrint Archive

License: CC BY 4.0

Abstract

Traditional security systems rely on the concealment of information through cryptographic transformation. In contrast, this paper reconstructs security from first principles, focusing on the *intent* of encryption rather than its historical form. We propose a new security primitive based on **indeterminate emergence**, wherein meaningful information or system effects do not exist outside authorized contexts and are indistinguishable from non-existence or benign noise to unauthorized observers.

We formalize this primitive using transcript-level indistinguishability over designer-specified hypothesis pairs, drawing on the Pufferfish privacy framework to define composable leakage bounds across adaptive interaction sequences. We introduce a three-plane model (observation, assumption, and action) and define a class of systems capable of real-world efficacy without information collapse. Central to this design is **absorptive execution**, a mode in which systems maintain computationally indistinguishable observational signals while silently nullifying effects under elevated inference risk.

We define explicit adversary models, prove security properties against each, provide a concrete instantiation for AI capability-set inference resistance, and position the framework against differential privacy, deniable encryption, ORAM, and zero-knowledge proofs. We analyze buildability, limits, and failure modes, and discuss applications in autonomous systems, AI governance, and secure pipelines.

1 Introduction

Security has historically been framed as the protection of representations: secrets are encoded, transmitted, and decoded under the assumption that adversaries may observe everything except keys. While effective for confidentiality, this model is poorly aligned with modern systems where the primary risk is not disclosure but **unintended capability emergence**, inference, or misuse.

This work asks:

> If we discard all existing definitions of encryption but retain its intent, what should a secure system look like?

Our answer replaces secrecy with **epistemic indeterminacy**: a formal guarantee that no feasible adversary can determine, through observation of a system's behavior, whether meaningful structure, capability, or effect exists.

Contributions. We make four contributions:

1. A formal security primitive, indeterminate emergence, defined over hypothesis-pair indistinguishability in the Pufferfish framework, with composable leakage bounds.
2. Absorptive execution: a degradation mode that enforces the leakage budget by silently nullifying effects while maintaining observational invariance.
3. Explicit adversary models (passive, active adaptive, insider, colluding endpoint) with per-model security guarantees.
4. A concrete instantiation demonstrating capability-set inference resistance for AI agent systems.

2 Reconstruction of Intent

The irreducible intent behind encryption is not scrambling symbols, but ensuring that:

> Artifacts may exist, move, and act in hostile environments such that only intended situations can produce meaning or effect.

This reframing reveals that:

- "Unreadable" is weaker than "non-existent." A ciphertext announces that a secret exists; indeterminate emergence does not.
- Silence can be informative. A system that refuses access leaks the existence of something worth refusing.
- Preventing *knowledge formation* may be more important than hiding data. Inference from behavioral patterns can be as damaging as key compromise.

3 System Model

3.1 Observation Plane

The observation plane defines everything an external observer can measure: outputs, timing, errors, silence, rates, and resource usage.

Let:

- $S \in \mathcal{S}$: hidden system state
- $a \in \mathcal{A}$: action taken by an external actor
- $O \in \mathcal{O}$: observable output

- $E \in \mathcal{E}$: effect space (world-changing outcomes)

The system is defined by:

Observation Channel:

$$O \sim C(O \mid S, a) \tag{1}$$

Effect Function:

$$E = F(S, a) \tag{2}$$

Security concerns the information content of O, not the internal structure of S.

Observation scope. We define the adversary's total observation at time t as:

$$O_t = (O_t^{sys}, O_t^{env}) \tag{3}$$

where O_t^{sys} captures system-emitted signals (responses, timing, resource signatures) and O_t^{env} captures environmentally mediated signals (downstream effects observable through the world). Security guarantees are parameterized by the adversary's accessible observation set; claims are scoped accordingly (see Section 6).

3.2 Assumptive Plane

Observers maintain beliefs over hypotheses H. Let $P_t(H)$ denote the belief distribution at time t, updated via:

$$P_{t+1}(H) = U(P_t(H), O_t, a_t) \tag{4}$$

We define **epistemic collapse** as:

$$\exists H^* \subset H_{sens} : P_t(H^*) \to 1 \tag{5}$$

where H_{sens} represents sensitive hypotheses. Importantly, certainty about *absence* is also information leakage: learning that a system definitely lacks a capability is itself a collapse event.

3.3 Action Plane

Attackers learn by acting. Penetration is active experimentation.

A secure system must allow real effects (efficacy) without producing learnable evidence. This yields the core constraint, formalized in the next section.

4 Formal Security Model

4.1 Secrets, Discriminators, and Adjacency

We adopt a Pufferfish-style framework [10] to define security over designer-specified hypothesis pairs rather than neighboring databases.

Secrets. Let $\mathcal{S}^* = \{s_i\}$ be a set of protected facts about the system (e.g., "tool X is in the capability set," "the system is in absorption mode," "policy P is active"). These are specified by the system designer.

Discriminative pairs. Let $\mathcal{D} = \{(s_i, s_j)\}$ be pairs of secrets the adversary should not be able to distinguish. For example: (tool X exists, tool X does not exist).

Observation mechanism. Let $\mathcal{M}$ be the mechanism that generates transcripts given secrets and actions:

$$T_n = (a_1, O_1, a_2, O_2, \ldots, a_n, O_n) \sim \mathcal{M}(s, a_{1:n}) \tag{6}$$

where the adversary adaptively selects a_t based on prior observations T_{t-1}.

4.2 Transcript Indistinguishability

Definition 4.1 (Epistemic Security)**.** *A system satisfies (ε, δ)-epistemic security with respect to secret set $\mathcal{S}^*$ and discriminative pairs $\mathcal{D}$ if for all $(s_i, s_j) \in \mathcal{D}$, all adaptive adversaries, and all measurable transcript sets $\mathcal{T}$:*

$$\Pr[T_n \in \mathcal{T} \mid s_i] \leq e^{\varepsilon} \Pr[T_n \in \mathcal{T} \mid s_j] + \delta \tag{7}$$

This definition provides:

- **Composability.** Standard and advanced composition theorems [6] apply directly. Under k-fold adaptive composition, the total privacy loss is bounded by $O(\sqrt{k \ln(1/\delta')} \cdot \varepsilon)$.
- **Operational meaning.** No hypothesis test applied to transcripts can distinguish s_i from s_j with advantage greater than $e^{\varepsilon} - 1 + \delta$.
- **Hypothesis-pair flexibility.** Unlike standard DP, the adjacency relation is domain-specific, defined by the designer over the secrets that matter.

4.3 Leakage Budget

Definition 4.2 (Leakage Budget)**.** *Each system context is assigned a total leakage budget $B = (\varepsilon_{total}, \delta_{total})$. The system maintains a privacy accountant that tracks cumulative leakage across interactions. At each step t, the interaction consumes $(\varepsilon_t, \delta_t)$ from the budget. The system enforces:*

$$\sum_{t=1}^{n} \varepsilon_t \leq \varepsilon_{total}, \quad 1 - \prod_{t=1}^{n} (1 - \delta_t) \leq \delta_{total} \tag{8}$$

(or tighter bounds under advanced composition).

When the budget is exhausted, the system transitions to absorption mode (Section 5.2).

4.4 Non-Collapse Guarantee

Theorem 4.1 (Non-Collapse)**.** *Under (ε, δ)-epistemic security with budget B, for all adaptive adversaries with query access bounded only by the budget, and for all discriminative pairs $(s_i, s_j) \in \mathcal{D}$:*

$$P(s_i \mid T_n) \leq \frac{e^{\varepsilon_{total}} P(s_i)}{e^{\varepsilon_{total}} P(s_i) + (1 - \delta_{total})(1 - P(s_i))} \tag{9}$$

where $P(s_i)$ is the adversary's prior. For small ε_{total} and δ_{total}, posterior concentration is bounded and epistemic collapse is prevented.

Proof sketch. Follows from the definition of transcript indistinguishability and Bayes' rule. The e^{ε} likelihood ratio bound limits how far the posterior can shift from the prior. □

5 Architecture

5.1 Sealed Capability Actuation

Efficacy is provided by a sealed capability actuator:

- **Intent Interface.** Accepts standardized action forms specifying *intent*, not implementation. The interface exposes a fixed, public action vocabulary.
- **Capability Token.** Encodes authorized action classes, rate limits, and budget allotments for the current session. Token contents are opaque to the interface.
- **Policy Verifier.** Checks the action request against the capability token and emits a fixed-format, non-informative receipt. All receipts have identical structure regardless of outcome.
- **Sealed Executor.** Interprets and acts using protected context. Operates within a defined isolation boundary (process isolation, TEE, or equivalent). The executor's internal state, memory access patterns, and computation are non-observable from outside the boundary.
- **Channel Shaper.** Enforces observational invariance: constant-rate output, fixed-size responses, bounded timing variance. Ensures O^{sys} is drawn from a state-independent distribution up to the tolerance η.
- **Effect Committer.** The single point at which effects leave the system boundary. Applies effects conditionally based on mode.

5.2 Absorptive Execution

Rather than denial or shutdown, systems enter **absorption mode** when the leakage budget is near exhaustion or under elevated inference risk.

Normal Mode:

$$O \sim Q(O \mid a), \quad E = F(S, a) \tag{10}$$

Absorption Mode:

$$O \sim Q(O \mid a), \quad E = \varnothing \tag{11}$$

Indistinguishability claim (computational). For any PPT distinguisher $\mathcal{D}$:

$$|\Pr[\mathcal{D}(O^{sys}_{1:n}) = 1 \mid \text{Normal}] - \Pr[\mathcal{D}(O^{sys}_{1:n}) = 1 \mid \text{Absorb}]| \leq \text{negl}(\lambda) \tag{12}$$

Indistinguishability claim (statistical, implementer's target). Under the Resource Trace Model (defined below):

$$D_{TV}\big(P(O^{sys} \mid \text{Normal}, h, a),\ P(O^{sys} \mid \text{Absorb}, h, a)\big) \leq \eta \tag{13}$$

Resource Trace Model. To make the indistinguishability claims meaningful, we specify what the adversary can observe about execution:

- **Level 1 (network-only):** Adversary sees response content and coarse timing (e.g., round-trip time at millisecond granularity). This is the weakest adversary and the easiest to satisfy.
- **Level 2 (system-level):** Adversary additionally observes system call traces, memory allocation patterns, and fine-grained timing.
- **Level 3 (co-located):** Adversary shares hardware and can observe cache timing, power consumption, or EM emanations.

Claims at Level 1 are achievable with standard channel shaping (fixed-size responses, padded timing, constant-rate I/O). Levels 2 and 3 require increasingly constrained execution environments (dedicated processes, TEEs) and are analogous to the challenges faced by constant-time cryptographic implementations [1, 4].

Engineering requirement. In absorption mode, the sealed executor must perform a *dummy computation* of equivalent resource profile to the real computation, not simply no-op. This ensures resource-trace indistinguishability at the appropriate level.

Key property. Absorption is the mechanism that enforces the leakage budget. Once the budget is exhausted, absorption ensures no further leakage (up to η). The budget is the policy; absorption is the enforcement; the composition theorem is the guarantee.

5.3 Absorption Transition and Meta-Leakage

The transition to absorption mode must itself be non-informative.

Budget uniformity. If different contexts receive different budgets based on sensitivity, the onset of absorption reveals sensitivity level. Two mitigations:

1. **Uniform budgets:** All contexts receive identical budgets regardless of actual sensitivity. Eliminates the inference but wastes budget on non-sensitive interactions.
2. **Randomized thresholds:** Absorption onset is stochastic: the system may enter absorption before budget exhaustion with some probability. The adversary observes absorption but cannot determine whether it was triggered by budget exhaustion or random early cutoff.

Stickiness. Once a context enters absorption, it remains absorbed until an out-of-band reset by an authorized party. This prevents adversaries from probing the absorption boundary by testing whether the system "comes back."

DoS consideration. An adversary who makes many queries can exhaust the budget and force absorption, effectively denying service. Mitigations include identity-bound budget partitioning (each identity gets an independent budget) and rate limiting at the intent interface. Note that identity binding is itself a design constraint: if identities are free, partitioning is ineffective. This represents a safety-over-liveness tradeoff that should be explicit in deployment.

6 Adversary Models and Security Claims

We define four adversary classes, in order of increasing power, and state which security properties hold against each.

6.1 Passive Transcript Observer

Capability. Observes O^{sys} only (network responses and coarse timing). Cannot inject actions.

Security claim. For all discriminative pairs $(s_i, s_j) \in \mathcal{D}$:

$$\Pr[T_n \in \mathcal{T} \mid s_i] \leq e^{\varepsilon} \Pr[T_n \in \mathcal{T} \mid s_j] + \delta \tag{14}$$

This is the strongest guarantee and follows directly from the observation channel design and channel shaping.

6.2 Active Adaptive Querier

Capability. Chooses actions a_t adaptively based on prior observations T_{t-1}. Bounded to n queries (or bounded by budget depletion, whichever comes first).

Security claim. Transcript indistinguishability holds under adaptive composition. After budget exhaustion, absorption ensures:

$$I(H_{sens}; O_t^{sys} \mid T_{t-1}, a_t) \leq \eta \quad \forall t > t_{absorb} \tag{15}$$

Note on replay. If the adversary replays the same action to average out noise, each replay is a query that consumes budget. The adversary trades budget for variance reduction. The composition theorem accounts for this: the total leakage is bounded regardless of whether queries are novel or repeated. Replay detection is therefore not strictly necessary for security, though it may be desirable for efficiency (preserving budget for legitimate interactions).

6.3 Insider with Partial Context

Capability. Observes all signals outside the sealed executor boundary: intent interface inputs, policy verifier decisions, capability token metadata (but not token contents), timing of commits, and any logs produced by components outside the executor.

Security claim. Indistinguishability holds for secrets whose distinguishing information is contained entirely within the sealed executor. The insider *may* learn whether absorption has occurred (depending on the execution model) and *may* learn capability token metadata. These leakages are explicitly scoped:

$$I(H_{sens}^{exec}; O_t^{insider} \mid T_{t-1}) \leq \varepsilon_{insider} \tag{16}$$

where H_{sens}^{exec} denotes secrets fully contained within the executor, and $\varepsilon_{insider}$ is quantified for the specific boundary implementation.

Residual risk. Secrets that span the executor boundary (e.g., the existence of a capability token for a specific action class, if token metadata reveals this) are not protected against insiders. This is a fundamental limit, not a failure of the framework.

6.4 Colluding Endpoint

Capability. The entity that receives effects (API, database, physical actuator) is adversarial or compromised. It can observe whether real effects arrived and correlate with action timing.

Security claim. The framework **does not** protect against this adversary for effect-existence secrets. If the adversary receives effects, it knows effects occurred. This is a fundamental limit: you cannot hide effects from their recipient.

However, the framework *does* protect all non-endpoint observers: for any party whose observation set does not include O^{env} at the effect endpoint, the full transcript indistinguishability guarantee holds.

Formal scoping. For adversary $\mathcal{A}$ with observation set $\mathcal{O}_{\mathcal{A}}$:

- If $O^{env}_{endpoint} \notin \mathcal{O}_{\mathcal{A}}$: full (ε, δ)-epistemic security holds.
- If $O^{env}_{endpoint} \in \mathcal{O}_{\mathcal{A}}$: only O^{sys}-based properties hold; effect existence is revealed.

7 Worked Example: AI Capability-Set Inference Resistance

We instantiate the framework for a concrete and timely problem: preventing an external observer from determining which tools an AI agent has access to.

7.1 Setting

An AI agent operates with access to a tool registry (file system, web APIs, code execution, database queries). An external observer watches the agent's API calls and outputs. The adversary's goal is to determine the agent's **capability set**—which tools are authorized for the current session.

Why this matters. Knowledge of the capability set enables targeted attacks: social engineering the agent toward a capability it has but shouldn't use in context, inferring organizational security posture from capability footprint, or identifying high-value targets by capability profile.

7.2 Mapping to Framework

7.3 Observation Channel Design

The critical design requirement is that the following three scenarios produce indistinguishable observations:

1. **Tool not available:** Agent requests tool X, which is not in the capability set.
2. **Tool available, request denied by policy:** Tool X is available, but the current request violates a policy constraint.

Framework Component	**Instantiation**
Secrets $\mathcal{S}^*$	Which tools are in the capability set
Discriminative pairs $\mathcal{D}$	(tool X present, tool X absent) for each tool X
Intent Interface	Natural-language action requests from the agent
Capability Token	Session-specific tool authorization list + budget
Policy Verifier	Checks request against token; emits fixed-format receipt
Sealed Executor	Invokes tool or enters absorption
Channel Shaper	Fixed-size responses; padded latency; constant-rate output
Effect Committer	Passes tool output to agent (or returns null result)

Table 1: Mapping of framework components to the AI capability-set inference resistance instantiation.

3. **Tool available, executed, returned empty:** Tool X is available, request is permitted, execution occurs, but the result is empty or null.

For all three cases, the observation channel emits:

$$O = (\texttt{receipt}_{fixed}, \texttt{payload}_{null}, \texttt{timing}_{padded}) \tag{17}$$

where `receipt_fixed` has identical structure across all outcomes, `payload_null` is a fixed-size null result indistinguishable from an empty tool response, and `timing_padded` is drawn from a fixed latency distribution.

7.4 Indistinguishability Argument

For a passive observer: all three scenarios produce identical observation distributions by construction. The observer's transcript is drawn from the same distribution regardless of which tools are in the capability set.

For an active adaptive querier: each query consumes budget. The adversary can submit action requests designed to probe for specific tools, but each probe reveals at most ε bits about the tool's presence (bounded by the per-interaction leakage). After n probes, total leakage is bounded by composition.

For the colluding endpoint: if the tool's target endpoint (e.g., an external API) is adversarial, it can observe whether a real API call arrived. This reveals tool presence to *that endpoint*. All other observers retain full indistinguishability guarantees.

7.5 Absorption in Practice

When the budget for a session is near exhaustion (e.g., an adversary has made many probing requests), the system enters absorption:

- Requests continue to receive fixed-format receipts and null payloads.
- No tool invocations occur.

- The agent receives plausible null results and must handle gracefully.

The agent's behavior in absorption may itself be informative (e.g., it may change its strategy). This is a known limitation: the framework protects the *system's* observation channel, not the *agent's* downstream behavior. Protecting agent behavior is a separate research problem.

7.6 Evaluation Plan

To validate the framework empirically:

1. **Divergence measurement.** For each discriminative pair, sample transcripts under both hypotheses and estimate D_{TV} and D_{KL} between transcript distributions. Verify that measured divergence falls within the theoretical bound η.
2. **Distinguisher evaluation.** Train a binary classifier on transcripts labeled by tool presence/absence. Measure classifier advantage. Verify advantage is bounded by $e^{\varepsilon_{total}} - 1 + \delta_{total}$.
3. **Budget depletion test.** Simulate an adaptive adversary executing an optimal probing strategy. Measure the number of queries required to achieve a target posterior concentration. Verify this exceeds the budget.

8 Side Channels and Flux

Side channels arise from emergent coupling across layers, what we term "flux." Rather than eliminating all leakage (impossible), we:

1. **Define a dominant, state-independent observation process** (the channel shaper) that produces the bulk of the adversary's observations.
2. **Drown residual signals** by ensuring that side-channel leakage is small relative to the dominant process noise.
3. **Bound adversary adaptivity** through the leakage budget, which limits the number of observations available for side-channel exploitation.

Security thus becomes a problem of **information geometry** (shaping the observation manifold so that sensitive hypotheses are indistinguishable) rather than a problem of perfect secrecy.

Effect-observation closure. When effects re-enter the observation channel through the environment (e.g., the adversary queries a database that the system wrote to), O^{env} expands. Our guarantees are explicitly scoped to the adversary's accessible observation set $\mathcal{O}_{\mathcal{A}}$. If O^{env} includes downstream effects, deniability about effect existence is impossible for that adversary. The framework's guarantees hold for all observers whose observation set does not include the effect endpoint.

9 Privacy Accountant

The system's leakage budget is enforced by a **privacy accountant**, not a heuristic anomaly detector. This design choice eliminates most failure modes associated with threshold-based monitoring.

9.1 Accountant Design

Inputs. At each interaction t, the accountant receives:

- The per-interaction leakage cost $(\varepsilon_t, \delta_t)$, determined by the observation channel mechanism and action type.
- The cumulative budget consumed so far.

Decision function. The accountant maintains a running total and triggers absorption when:

$$\sum_{t'=1}^{t} \varepsilon_{t'} \geq \varepsilon_{total} - \varepsilon_{margin} \tag{18}$$

where ε_{margin} is a safety margin. Under randomized thresholds, the trigger point is drawn from a distribution over $[\varepsilon_{total} - \varepsilon_{margin}, \varepsilon_{total}]$ at session start.

Transition policy. Absorption is sticky: once triggered, the context remains absorbed until out-of-band reset.

9.2 Accountant Properties

- **No oracle risk.** The accountant is a counter, not a classifier. Its behavior is deterministic given the interaction history and does not depend on the content of sensitive hypotheses.
- **Meta-leakage.** The accountant's state (how much budget remains) is not directly observable, but the *onset of absorption* may be indirectly observable if the adversary detects behavioral changes. Mitigation: randomized thresholds or uniform budgets (Section 5.3).
- **DoS.** Addressable through identity-bound budget partitioning and rate limiting.

9.3 Relationship to Heuristic Monitoring

In some deployments, a heuristic monitor (anomaly detection, action entropy scoring) may supplement the accountant to detect adversarial behavior *before* significant budget is consumed. However, the security guarantees rest on the accountant alone; the heuristic monitor is an efficiency optimization, not a security mechanism.

10 Considered and Rejected Approaches

Metadata-as-Encryption. Rejected: metadata must be readable; encrypting it creates a new metadata layer.

Context-as-Key. Rejected: context is replicable without physical or cryptographic anchors.

Obfuscation. Rejected: collapses under inspection; security-through-obscurity.

Pure Cryptography (as sufficient). Rejected: meaning always exists in ciphertext; key compromise is catastrophic and retroactive. Indeterminate emergence avoids retroactive collapse because meaning never exists outside the authorized context, so there is nothing to retroactively reveal.

Each of these approaches informed the final model but failed to meet the reconstructed intent.

11 Comparison to Classical Cryptography

We compare indeterminate emergence to classical cryptography along three dimensions:

Adversary advantage. Under standard assumptions, a PPT adversary's advantage against a well-implemented cryptosystem is negligible in the security parameter. Under indeterminate emergence, the adversary's advantage is bounded by $e^{\varepsilon_{total}} - 1 + \delta_{total}$, which is non-negligible but tunable and explicit.

Failure mode. Classical crypto fails catastrophically on key compromise: the adversary gains access to all past and current plaintexts. Indeterminate emergence degrades monotonically: budget exhaustion triggers absorption, which halts further leakage. There is no single point of failure that retroactively reveals past interactions.

What the adversary learns on success. A successful attack on classical crypto reveals the plaintext: everything, all at once. A "successful" probing campaign against an indeterminate emergence system yields bounded, forward-only information: the adversary may increase posterior probability on some hypotheses, but never gains certainty, and past observations cannot be retroactively reinterpreted.

The tradeoff. Indeterminate emergence trades per-query security strength for graceful degradation and absence of retroactive collapse. This tradeoff is favorable in settings where the threat is behavioral inference rather than data exfiltration, and where the consequence of partial leakage is manageable but catastrophic disclosure is not.

12 Related Work

12.1 Differential Privacy

Differential privacy [5] provides composable indistinguishability guarantees for database queries. Our framework adopts DP's composition machinery but applies it to action transcripts rather than query answers, and uses Pufferfish-style [10] hypothesis-pair adjacency rather than neighboring-database adjacency.

12.2 Deniable Encryption and Steganography

Deniable encryption [3] ensures that a ciphertext is consistent with multiple possible plaintexts, preventing coercion. Steganography hides the existence of communication within innocuous

cover traffic. Our framework extends deniability from stored data to **real-time effects and capabilities**: the system doesn't just deny what message was sent, but whether any meaningful action was taken.

12.3 Oblivious RAM

ORAM [8] hides memory access patterns from an adversary observing storage. Our channel shaper serves an analogous function for the observation plane, ensuring that observable behavior is independent of system state. The key difference is that ORAM protects data access patterns within a computation, while our framework protects the existence and nature of the computation itself.

12.4 Zero-Knowledge Proofs

ZK proofs [9] allow proving a statement without revealing why it's true. Our policy verifier's fixed-format receipts serve a similar function: the receipt proves the request was processed without revealing the policy decision or the capabilities consulted.

12.5 Secure Multi-Party Computation

MPC ensures that intermediate values never exist in any single location. This aligns with our principle that meaning should never exist outside the sealed executor: just as MPC distributes a secret across parties, indeterminate emergence confines meaning to an authorized execution context.

12.6 Cyber Deception and Honeypots

Honeypots [11] are systems that appear functional but produce no real effects, designed to be indistinguishable from real systems. Absorptive execution has structural similarity: it produces observations without real effects. The key difference is that honeypots are static deceptions (separate systems), while absorption is a dynamic mode that the *same system* enters and exits. Our framework provides formal guarantees on the mode transition's indistinguishability, which honeypots typically lack.

12.7 Novelty Summary

We unify transcript indistinguishability (from DP), deniability of existence (from deniable encryption), access pattern hiding (from ORAM), proof without revelation (from ZK), and non-existence of intermediate values (from MPC) into a single action-centric framework. The novel primitive is absorptive execution, which extends deniability from stored data to real-time effects and capabilities, enforced by a composable leakage budget.

13 Buildability

No new cryptographic primitives are required; however, the composition of capability systems, sealed execution, channel shaping, and privacy accounting is novel and demands rigorous spec-

ification and evaluation.

The system is buildable using:

- **Capability token systems** for action authorization (e.g., Macaroons [2], UCAN).
- **Process isolation or TEEs** for sealed execution (e.g., SGX, SEV, or application-level sandboxing).
- **Constant-signature interfaces** for channel shaping (fixed-size responses, padded timing via techniques from traffic analysis resistance [7]).
- **Privacy accounting libraries** for budget tracking (e.g., Google's DP library, Opacus).
- **Policy engines** for action verification (e.g., OPA, Cedar).

The integration and invariance enforcement across these components is where the implementation challenge lies, and each instantiation must be evaluated against the Resource Trace Model (Section 5.2) for the target deployment.

14 Limits

Absolute security is impossible. Specific limits of this framework include:

1. **Colluding endpoint.** Effect existence cannot be hidden from the entity that receives effects.
2. **Agent behavioral leakage.** The framework protects the system's observation channel, not the downstream behavior of agents that consume system outputs.
3. **Modeling assumptions.** Guarantees depend on the designer's specification of $\mathcal{S}^*$ and $\mathcal{D}$. Misspecified secrets or discriminative pairs leave blind spots.
4. **Non-copyable anchors.** Context-as-key approaches fail without physical or cryptographic anchors. The framework does not create new anchors; it works within existing trust boundaries.
5. **Dynamic sensitivity.** H_{sens} is treated as a design parameter. Adaptive or context-dependent sensitivity classification is future work.
6. **Budget-liveness tradeoff.** Adversaries can force absorption by exhausting budgets, creating a denial-of-service vector mitigated but not eliminated by identity partitioning.

15 Applications

- **AI agent governance:** Preventing capability-set inference and unauthorized capability discovery.
- **Autonomous toolchains:** Allowing agents to act without revealing what they can do.

- **Secure CI/CD:** Pipelines that execute deployments without revealing infrastructure topology.
- **Capability containment:** Systems where the existence of a capability is itself sensitive.
- **Policy enforcement:** Environments where denial signals are as informative as access.
- **Adversarial ML defense:** Preventing model probing from revealing training data properties or architectural details.

16 Conclusion

By reconstructing security from intent rather than tradition, we arrive at a system where meaning does not need to be hidden because it does not exist outside authorized contexts. Indeterminate emergence provides strong, composable, and governance-aligned security properties suitable for modern autonomous systems.

The framework is defined over explicit adversary models, provides composable leakage bounds via Pufferfish-style transcript indistinguishability, degrades gracefully through absorptive execution, and is instantiable with existing components. Its limits (colluding endpoints, agent behavioral leakage, and the modeling burden of specifying secrets) are fundamental and explicitly scoped rather than hidden.

The strongest security signal is not silence, but the inability to determine whether silence is meaningful, accidental, or impossible.

A Formal Definitions and Proofs

A.1 Notation Summary

Symbol	Meaning
$\mathcal{S}^*$	Protected secrets (designer-specified)
$\mathcal{D}$	Discriminative pairs
T_n	Transcript of n interactions
(ε, δ)	Per-interaction leakage parameters
$B = (\varepsilon_{total}, \delta_{total})$	Leakage budget
η	Absorptive mode observational tolerance
λ	Computational security parameter
O^{sys}, O^{env}	System and environment observation components
H_{sens}	Sensitive hypotheses

Table 2: Notation summary.

A.2 Proof of Theorem 4.1 (Non-Collapse)

By Definition 4.1, for any transcript set $\mathcal{T}$ and discriminative pair (s_i, s_j):

$$\Pr[T_n \in \mathcal{T} \mid s_i] \leq e^{\varepsilon_{total}} \Pr[T_n \in \mathcal{T} \mid s_j] + \delta_{total} \tag{19}$$

By Bayes' rule, the posterior ratio satisfies:

$$\frac{P(s_i \mid T_n)}{P(s_j \mid T_n)} = \frac{P(T_n \mid s_i)}{P(T_n \mid s_j)} \cdot \frac{P(s_i)}{P(s_j)} \leq e^{\varepsilon_{total}} \cdot \frac{P(s_i)}{P(s_j)} \tag{20}$$

(ignoring δ_{total} for the clean bound; the full bound incorporates δ via standard techniques). For uniform prior $P(s_i) = P(s_j) = 1/2$:

$$P(s_i \mid T_n) \leq \frac{e^{\varepsilon_{total}}}{1 + e^{\varepsilon_{total}}} \tag{21}$$

For $\varepsilon_{total} = 1$: $P(s_i \mid T_n) \leq 0.731$. For $\varepsilon_{total} = 0.1$: $P(s_i \mid T_n) \leq 0.525$.
Epistemic collapse ($P(s_i \mid T_n) \to 1$) is prevented for any finite budget. □

A.3 Composition Theorem (Standard)

Under k adaptive interactions each satisfying (ε, δ)-epistemic security:

$$(\varepsilon_{total}, \delta_{total}) = (k\varepsilon, k\delta) \quad \text{(basic composition)} \tag{22}$$

Under advanced composition [6]:

$$\varepsilon_{total} = \sqrt{2k \ln(1/\delta')} \cdot \varepsilon + k\varepsilon(e^{\varepsilon} - 1), \quad \delta_{total} = k\delta + \delta' \tag{23}$$

for any $\delta' > 0$.

A.4 Absorptive Mode Indistinguishability

Under the Resource Trace Model at Level 1, absorptive mode satisfies the statistical claim if:

1. Response content is drawn from the same fixed distribution $Q(O \mid a)$ in both modes.
2. Response timing is drawn from a fixed latency distribution Λ independent of mode.
3. Response size is constant.

Conditions 1–3 are achievable by construction (fixed-format receipts, constant-size payloads, padded timing). Under these conditions, $D_{TV}(P(O^{sys} \mid \text{Normal}), P(O^{sys} \mid \text{Absorb})) = 0$ at Level 1.

At Levels 2 and 3, the claim depends on the executor's ability to match resource profiles between real and dummy computation, which is an implementation-specific guarantee.

B Reference Architecture

C Intended Use

This paper supports implementation, verification, and audit of systems designed for:

- AI agent governance and capability containment

- Secure action pipelines with inference resistance
- Inference security research
- Systems where the existence of capability is itself sensitive

References

[1] Daniel J. Bernstein. Cache-timing attacks on AES. Technical report, 2005. Available at: https://cr.yp.to/antiforgery/cachetiming-20050414.pdf.

[2] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *Proceedings of the Network and Distributed System Security Symposium (NDSS 2014)*. Internet Society, 2014.

[3] Ran Canetti, Cynthia Dwork, Moni Naor, and Rafail Ostrovsky. Deniable encryption. In *Advances in Cryptology — CRYPTO '97*, volume 1294 of *Lecture Notes in Computer Science*, pages 90–104. Springer, 1997. doi: 10.1007/BFb0052229.

[4] Bart Coppens, Ingrid Verbauwhede, Koen De Bosschere, and Bjorn De Sutter. Practical mitigations for timing-based side-channel attacks on modern x86 processors. In *Proceedings of the 2009 30th IEEE Symposium on Security and Privacy*, pages 45–60. IEEE, 2009.

[5] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. Calibrating noise to sensitivity in private data analysis. In *Proceedings of the 3rd Theory of Cryptography Conference (TCC 2006)*, volume 3876 of *Lecture Notes in Computer Science*, pages 265–284. Springer, 2006.

[6] Cynthia Dwork, Guy N. Rothblum, and Salil Vadhan. Boosting and differential privacy. In *Proceedings of the 51st Annual IEEE Symposium on Foundations of Computer Science (FOCS 2010)*, pages 51–60. IEEE, 2010. doi: 10.1109/FOCS.2010.12.

[7] Kevin P. Dyer, Scott E. Coull, Thomas Ristenpart, and Thomas Shrimpton. Peek-a-boo, I still see you: Why efficient traffic analysis countermeasures fail. In *Proceedings of the 2012 IEEE Symposium on Security and Privacy (S&P 2012)*, pages 332–346. IEEE, 2012.

[8] Oded Goldreich and Rafail Ostrovsky. Software protection and simulation on oblivious rams. *Journal of the ACM*, 43(3):431–473, 1996.

[9] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof-systems. In *Proceedings of the 17th Annual ACM Symposium on Theory of Computing (STOC 1985)*, pages 291–304. ACM, 1985.

[10] Daniel Kifer and Ashwin Machanavajjhala. Pufferfish: A framework for mathematical privacy definitions. *ACM Transactions on Database Systems*, 39(1):3:1–3:36, 2014. doi: 10.1145/2514689.

[11] Lance Spitzner. *Honeypots: Tracking Hackers*. Addison-Wesley Professional, 2003.

```
+-----------------------------------------------------------------+
|                    External Actor                               |
|                         |                                       |
|                   Action Request                                |
|                         v                                       |
|              +----------------------+                           |
|              |   Intent Interface   |  Fixed action vocabulary  |
|              +----------+----------+                            |
|                         |                                       |
|                         v                                       |
|              +----------------------+                           |
|              |  Capability Token    |  Opaque to interface      |
|              |      Check           |                           |
|              +----------+----------+                            |
|                         |                                       |
|                         v                                       |
|              +----------------------+    +------------------+   |
|              |   Policy Verifier    |--->|  Fixed Receipt   |   |
|              |                      |    |  (to actor)      |   |
|              +----------+----------+    +------------------+   |
|                         |                                       |
|                +--------+--------+                               |
|                v                 v                                |
|   +-------------------+  +---------------+                      |
|   |  Sealed Executor  |  |   Privacy     |                      |
|   |  (isolation       |  |  Accountant   |                      |
|   |   boundary)       |  |  (budget      |                      |
|   |                   |  |   tracking)   |                      |
|   |  Normal: execute  |  +-------+-------+                      |
|   |  Absorb: dummy    |<---------+                              |
|   +--------+----------+  mode signal                            |
|            |                                                     |
|            v                                                     |
|   +---------------------+                                        |
|   |   Channel Shaper    |  Constant rate, fixed size,            |
|   |                     |  bounded timing variance               |
|   +----------+----------+                                        |
|              |                                                   |
|              v                                                   |
|   +---------------------+                                        |
|   |  Effect Committer   |---------->  External World             |
|   |  (conditional)      |            (effects or null)           |
|   +---------------------+                                        |
+-----------------------------------------------------------------+
```

Figure 1: Reference architecture for an indeterminate emergence system.